Project Overview — Simple Mail Server & Client

1. Architecture Description

The project implements a simple text-based mail system using a client–server model over TCP sockets. The communication follows a minimal custom line-based protocol inspired by SMTP-like semantics.

Server

- The server is a **multi-threaded**, blocking TCP server written in modern C++ (C++20).
- It listens for incoming connections on a configurable port (`6543` by default).
- The main thread only accepts clients; each connection is then handled by a worker from a **thread pool**.
- Every worker runs an independent command–response loop for its client.
- Supported commands:
  - `LOGIN` – authenticate a user against the internal LDAP server
  - `SEND` – store a new mail for the authenticated user
  - `LIST` – list all stored messages of the authenticated user
  - `READ` – retrieve the full content of a specific message
  - `DEL` – delete a specific message
  - `QUIT` – close the connection
- **Authentication is mandatory**: all commands except `QUIT` are rejected until a successful `LOGIN` has been completed.
- Mails are stored as plain text files under a local `mail-spool/<username>/` directory, with filenames `<id>.txt`.
- Each message file contains simple headers:

```
FROM: <sender>
SUBJECT: <subject>

<message body>
```

- The server performs input validation and uses helper functions (`is_valid_username`, `is_valid_subject`, `parse_message_id`) to ensure protocol integrity.
- Concurrency safety is ensured using **per-user mutexes** (one mutex per mailbox directory) so concurrent operations on the same inbox do not race.
- Responses are simple control lines like `OK\n` or `ERR\n`, keeping the protocol human-readable.

LDAP Authentication (`LOGIN`)

- The login protocol is line-based:

```
LOGIN
<LDAP username>
<password in plain text>
```

- The server builds a DN of the form `uid=<username>,ou=people,dc=technikum-wien,dc=at` and authenticates via **TLS-secured LDAP bind** (`ldap_start_tls_s` + `ldap_sasl_bind_s`).
- On success, the session becomes authenticated and the username is stored for further commands.
- On failure, the server responds with `ERR` and records a failed attempt for the client IP.

Blacklist / Login Attempt Limiting

- For security reasons **only 3 failed login attempts per IP** are allowed.
- After 3 failures the client's IP address is **blacklisted for 1 minute**.
- While blacklisted, any LOGIN attempt is rejected with:

```
ERR blacklisted
```

- The blacklist is **persisted to disk** (e.g., `blacklist.txt`) and reloaded on startup.
- Expired blocks are automatically removed when checked.

Client

- The client is a console-based interactive program, also implemented in C++.
- It connects to the server via TCP, sends commands, and displays responses.
- It supports full interaction:
  - `LOGIN`: prompts for username and hidden password (termios-based `getpass()` on a real terminal).
  - `SEND`: prompts for receiver, subject, and message body.
  - `LIST`: displays the subject list for the authenticated user.
  - `READ`: displays the full content of a stored message.
  - `DEL`: deletes a stored message by id.
  - `QUIT`: exits cleanly.
- Communication is line-oriented (newline terminated) to maintain cross-platform protocol compatibility.

2. Used Technologies

Category	Technology	Purpose
Language	C++20	Core implementation for both client and server

Category	Technology	Purpose
Standard Libraries	`<filesystem>`, `<fstream>`, `<sstream>`, `<thread>`, `<mutex>`, `<chrono>`	File management, threading, synchronization, blacklist timing
Networking	POSIX sockets (`<sys/socket.h>`, `<netinet/in.h>`, `<arpa/inet.h>`)	TCP server/client communication
LDAP	OpenLDAP (`libldap`, `liblber`)	User authentication
Build System	Make / `g++`	Simple and portable compilation
Platform	POSIX / Linux	Uses standard Unix socket & termios APIs

3. Development Strategy and Adaptations

Development Strategy

The project was built in incremental, test-driven stages:

1. Base server setup: implemented socket creation, binding, listening, and connection handling.
2. Core command `SEND`: introduced message writing and storage using `<filesystem>`.
3. Protocol helpers: extracted common validation and socket reading logic into modular helper files (`readLineFromSocket.cpp`, `protocolValidation.cpp`).
4. Client prototype: built an interactive CLI client to manually test `SEND` and `QUIT`.
5. Command extensions: added `LIST`, `READ`, and `DEL` handlers, following the same modular design and comment conventions.
6. **Thread pool integration**: introduced a worker pool to serve multiple clients concurrently.
7. **Authentication extension**: added `LOGIN`, LDAP bind, and per-connection authentication state.
8. **Blacklist extension**: added per-IP login attempt counting, 60s blocking, and persistence.
9. Testing & cleanup: verified complete round-trip client/server communication with multiple parallel clients, ensured protocol compliance, and unified code style and documentation.

Needed Adaptations

- Protocol adjustments: refined the format of responses to stay line-based and human-readable.
- File system organization: introduced per-user folders and incremental numeric message IDs.
- Robustness improvements: added error handling for socket interruptions (`EINTR`) and invalid user/message inputs.
- **Concurrency support**: moved from single-threaded handling to a thread-pool model and added per-user mutexes.
- **Security improvements**: enforced LDAP-based `LOGIN` before mailbox access and added a persistent blacklist for brute-force protection.
- Consistency: standardized logging, function naming, and comment structure across all modules.
- Testing tooling: upgraded the client to fully exercise all commands interactively, enabling complete end-to-end testing.